Experiment 1: Flight Reservation System

Program (Python 3)

```
flights = [
    ("AI101", "Chennai", "Delhi", "06:00", "Domestic", 5200),
    ("6E202", "Chennai", "Mumbai", "09:30", "Domestic", 4100),
    ("EK544", "Chennai", "Dubai", "03:15", "International", 18500),
    ("SQ403", "Chennai", "Singapore", "23:50", "International", 21000),
]
class_mult = {"Economy": 1.0, "Premium Economy": 1.4, "Business": 2.2}
fare_mult = {"Regular": 1.0, "Student": 0.9, "Senior Citizen": 0.85, "Armed Forces": 0.8}

def search(trip, frm, to, dep, ret, trav, cls, fare, scope):
    print("\n------ FLIGHT SEARCH ------")
    print("Trip Type :", trip, "| Scope:", scope)
    print("From:", frm, " To:", to)
    print("Departure:", dep, " Return:", ret if trip == "Round Trip" else "N/A")
    print("Travelers:", trav, "| Class:", cls, "| Fare Type:", fare)
    found = False
    for f in flights:
        if f[1].lower() == frm.lower() and f[2].lower() == to.lower() and f[4] == scope:
            found = True
            per = f[5] * class_mult[cls] * fare_mult[fare]
            total = per * trav * (2 if trip == "Round Trip" else 1)
            print(f"Flight {f[0]} dep {f[3]}  Fare/person: Rs.{per:.2f}  TOTAL: Rs.{total:.2f}")
    if not found:
        print("No flights available for this route.")

search("One Way", "Chennai", "Delhi", "2026-11-10", "", 2, "Economy", "Regular", "Domestic")
search("Round Trip", "Chennai", "Dubai", "2026-12-01", "2026-12-15", 3, "Business", "Student", "International")
```

Output

```
------ FLIGHT SEARCH ------
Trip Type : One Way | Scope: Domestic
From: Chennai  To: Delhi
Departure: 2026-11-10  Return: N/A
Travelers: 2 | Class: Economy | Fare Type: Regular
Flight AI101 dep 06:00  Fare/person: Rs.5200.00  TOTAL: Rs.10400.00

------ FLIGHT SEARCH ------
Trip Type : Round Trip | Scope: International
From: Chennai  To: Dubai
Departure: 2026-12-01  Return: 2026-12-15
Travelers: 3 | Class: Business | Fare Type: Student
Flight EK544 dep 03:15  Fare/person: Rs.36630.00  TOTAL: Rs.219780.00
```

Reference output generated by running the program above (Python 3.13 / gcc). Not a screenshot of an online compiler.

Experiment 2: Property Buying & Rental (Chennai)

Program (Python 3)

```
properties = [
    {"id": 1, "type": "Buy", "cat": "Residential", "area": "Adyar", "value": 9500000},
    {"id": 2, "type": "Rent", "cat": "Residential", "area": "T Nagar", "value": 25000},
    {"id": 3, "type": "Rent", "cat": "Commercial", "area": "Guindy", "value": 80000},
    {"id": 4, "type": "Buy", "cat": "Commercial", "area": "OMR", "value": 22000000},
    {"id": 5, "type": "Buy", "cat": "Residential", "area": "Tambaram", "value": 4800000},
]
def search(ptype, cat, area, lo, hi):
    print(f"\nSearch -> {ptype} | {cat} | Area: {area or 'Any'} | Range: {lo}-{hi}")
    n = 0
    for p in properties:
        if p["type"] == ptype and p["cat"] == cat and (not area or p["area"] == area) and lo <= p["value"] <= hi:
            print(f"ID {p['id']}: {p['cat']} property at {p['area']}, Chennai - Rs.{p['value']}")
            n += 1
    if n == 0: print("No matching properties.")

def rental_agreement(tenant, owner, p_id, months, deposit):
    p = next(x for x in properties if x["id"] == p_id)
    print("\n===== RENTAL AGREEMENT =====")
    print(f"Owner: {owner} | Tenant: {tenant}")
    print(f"Property: {p['cat']} at {p['area']}, Chennai")
    print(f"Monthly Rent: Rs.{p['value']} | Period: {months} months | Deposit: Rs.{deposit}")
    print(f"Total Rent Payable: Rs.{p['value']*months}")
    print("Signed digitally. Agreement generated successfully.")

search("Buy", "Residential", "", 1000000, 10000000)
search("Rent", "Commercial", "Guindy", 10000, 100000)
rental_agreement("Arun Kumar", "Ravi Shankar", 2, 11, 100000)
```

Output

```
Search -> Buy | Residential | Area: Any | Range: 1000000-10000000
ID 1: Residential property at Adyar, Chennai - Rs.9500000
ID 5: Residential property at Tambaram, Chennai - Rs.4800000

Search -> Rent | Commercial | Area: Guindy | Range: 10000-100000
ID 3: Commercial property at Guindy, Chennai - Rs.80000

===== RENTAL AGREEMENT =====
Owner: Ravi Shankar | Tenant: Arun Kumar
Property: Residential at T Nagar, Chennai
Monthly Rent: Rs.25000 | Period: 11 months | Deposit: Rs.100000
Total Rent Payable: Rs.275000
Signed digitally. Agreement generated successfully.
```

Reference output generated by running the program above (Python 3.13 / gcc). Not a screenshot of an online compiler.

Experiment 3: Car Booking Reservation System

Program (Python 3)

```
cabs = [("Mini", 11, 700), ("Sedan", 14, 900), ("SUV", 18, 1300)]   # name, Rs/km, Rs/hr package
def book(service, frm, to, pkg, qty, cab_idx, km):
    name, per_km, per_hr = cabs[cab_idx]
    print("\n------ CAB BOOKING ------")
    print("Service:", service, "| From:", frm, "| To:", to or "N/A")
    print("Cab:", name)
    if service == "Rental":
        fare = per_hr * qty
        print(f"Package: {pkg} ({qty} hours)  Fare: Rs.{fare}")
    elif service == "Outstation":
        fare = per_km * km * 2 if pkg == "Round" else per_km * km
        print(f"Package: {pkg} trip, {km} km  Fare: Rs.{fare}")
    else:
        fare = per_km * km
        print(f"Local ride {km} km  Fare: Rs.{fare}")
    print("Booking Confirmed! Total Fare = Rs.", fare)

book("Local", "Saveetha", "Airport", "", 0, 0, 25)
book("Rental", "Chennai", "", "4hr/40km", 4, 1, 0)
book("Outstation", "Chennai", "Pondicherry", "Round", 1, 2, 150)
```

Output

```
------ CAB BOOKING ------
Service: Local | From: Saveetha | To: Airport
Cab: Mini
Local ride 25 km  Fare: Rs.275
Booking Confirmed! Total Fare = Rs. 275

------ CAB BOOKING ------
Service: Rental | From: Chennai | To: N/A
Cab: Sedan
Package: 4hr/40km (4 hours)  Fare: Rs.3600
Booking Confirmed! Total Fare = Rs. 3600

------ CAB BOOKING ------
Service: Outstation | From: Chennai | To: Pondicherry
Cab: SUV
Package: Round trip, 150 km  Fare: Rs.5400
Booking Confirmed! Total Fare = Rs. 5400
```

Reference output generated by running the program above (Python 3.13 / gcc). Not a screenshot of an online compiler.

Experiment 4: Library Book Reservation (SIMATS)

Program (Python 3)

```
books = [
    {"title": "Cloud Computing", "author": "Rajkumar Buyya", "pub": "Wiley", "year": 2013, "edition": 1, "copies": 5, "rack": "A12", "status": "returned"},
    {"title": "Database Systems", "author": "Elmasri", "pub": "Pearson", "year": 2015, "edition": 7, "copies": 3, "rack": "B05", "status": "returned"},
    {"title": "Big Data Analytics", "author": "Seema Acharya", "pub": "Wiley", "year": 2016, "edition": 2, "copies": 2, "rack": "C09", "status": "returned"},
]
def show():
    print(f"\n{'Title':20}{'Author':16}{'Pub':9}{'Year':6}{'Ed':4}{'Copies':8}{'Rack':6}{'Status'}")
    for b in books:
        print(f"{b['title']:20}{b['author']:16}{b['pub']:9}{b['year']:<6}{b['edition']:<4}{b['copies']:<8}{b['rack']:6}{b['status']}")
def issue(title):
    for b in books:
        if b["title"] == title:
            if b["copies"] > 0:
                b["copies"] -= 1; b["status"] = "taken"; print("\nIssued:", title)
            else: print("\nNo copies available")
def give_back(title):
    for b in books:
        if b["title"] == title:
            b["copies"] += 1; b["status"] = "returned"; print("\nReturned:", title)
show(); issue("Cloud Computing"); show(); give_back("Cloud Computing"); show()
```

Output

```
Title               Author          Pub      Year  Ed  Copies  Rack  Status
Cloud Computing     Rajkumar Buyya  Wiley    2013  1   5       A12   returned
Database Systems    Elmasri         Pearson  2015  7   3       B05   returned
Big Data Analytics  Seema Acharya   Wiley    2016  2   2       C09   returned

Issued: Cloud Computing

Title               Author          Pub      Year  Ed  Copies  Rack  Status
Cloud Computing     Rajkumar Buyya  Wiley    2013  1   4       A12   taken
Database Systems    Elmasri         Pearson  2015  7   3       B05   returned
Big Data Analytics  Seema Acharya   Wiley    2016  2   2       C09   returned

Returned: Cloud Computing

Title               Author          Pub      Year  Ed  Copies  Rack  Status
Cloud Computing     Rajkumar Buyya  Wiley    2013  1   5       A12   returned
Database Systems    Elmasri         Pearson  2015  7   3       B05   returned
Big Data Analytics  Seema Acharya   Wiley    2016  2   2       C09   returned
```

Reference output generated by running the program above (Python 3.13 / gcc). Not a screenshot of an online compiler.

Experiment 5: Mark Sheet Processing System

Program (Python 3)

```
subjects = ["Cloud", "BigData", "Maths", "Java", "DBMS"]
students = [
    ("Arun", "192211001", [88, 92, 79, 85, 90]),
    ("Divya", "192211002", [95, 89, 91, 94, 96]),
    ("Karthik", "192211003", [60, 55, 48, 70, 65]),
    ("Meena", "192211004", [72, 80, 68, 75, 79]),
]
result = []
for n, r, m in students:
    total = sum(m); avg = total / len(m); pct = total / (len(m) * 100) * 100
    result.append([n, r, m, total, avg, pct])
result.sort(key=lambda x: x[3], reverse=True)
def perf(p):
    return "Distinction" if p >= 75 else "First Class" if p >= 60 else "Second Class" if p >= 50 else "Pass/Fail"
print(f"{'Rank':5}{'Name':9}{'RegNo':11}" + "".join(f"{s:9}" for s in subjects) + f"{'Total':7}{'Avg':7}{'%':7}Performance")
for i, (n, r, m, t, a, p) in enumerate(result, 1):
    print(f"{i:<5}{n:9}{r:11}" + "".join(f"{x:<9}" for x in m) + f"{t:<7}{a:<7.1f}{p:<7.1f}{perf(p)}")
```

Output

```
Rank Name     RegNo      Cloud    BigData  Maths    Java     DBMS     Total  Avg    %      Performance
1    Divya    192211002  95       89       91       94       96       465    93.0   93.0   Distinction
2    Arun     192211001  88       92       79       85       90       434    86.8   86.8   Distinction
3    Meena    192211004  72       80       68       75       79       374    74.8   74.8   First Class
4    Karthik  192211003  60       55       48       70       65       298    59.6   59.6   Second Class
```

Reference output generated by running the program above (Python 3.13 / gcc). Not a screenshot of an online compiler.